Sujet examen Semestre 1 - Module Algorithmique & Programmation

TC101 - Python & JavaScript

Durée : 3 heures
Documents autorisés : Aucun
Matériel autorisé : Calculatrice simple, PC (Uniquement dans la partie 3)
Barème : /100 points

Instructions générales :

- Vous pouvez répondre en Python OU en JavaScript (précisez votre choix pour chaque exercice) Sauf si l'exercice exige un des deux langages ou les 2 langages.
- Écrivez du code clair et commenté
- Lisez attentivement toutes les consignes avant de commencer
- Gérez les cas limites et les erreurs
- Répondre directement sur les espaces réservés pour les réponses
- Lorsque vous arrivez à la partie 3; prévenez votre responsable pour qu'il/elle vous donne accès à votre PC

Nom(s) et Prénom(s) : _____

Date : _____

PARTIE 1 : QUESTIONS DE COURS (30 points)

Question 1.1 : Algorithmique et Représentation (8 points)

a) Qu'est-ce qu'un algorithme ? Donnez une définition simple et citez deux caractéristiques importantes. (4 points)

Réponse :

b) Expliquez la différence entre un organigramme et du pseudo-code pour représenter un algorithme. Donnez un exemple simple de chacun pour l'algorithme suivant : **"Vérifier si un nombre est positif, négatif ou nul"**

Réponse :

Pseudo-Code	Organigramme

GREEN UP ACADEMY

Question 1.2 : Types de données et Variables (7 points)

a) Donnez le type de chacune de ces valeurs Python et expliquez : (4 points)

- **42**
- **"42"**
- **3.14**
- **True**

Réponse :

b) Comment créer une liste et comment accéder à son premier élément en Python ? (3 points)

Réponse :

Question 1.3 : Complexité algorithmique (8 points)

Rappel sur la notation Big O :

- **O(1)** : Complexité constante (temps fixe)
- **O(log N)** : Complexité logarithmique (ex: recherche dichotomique)
- **O(N)** : Complexité linéaire (parcours complet)
- **O(N log N)** : Complexité quasi-linéaire (tri efficace)

- **$O(N^2)$** : Complexité quadratique (double boucle)

a) Analysez la complexité temporelle du code suivant et justifiez : (4 points)

```
def chercher(liste, element):
    for item in liste:
        if item == element:
            return True
    return False
```

Réponse : _____

Justification :

_____

_____

_____

b) Comparez la complexité des algorithmes de tri suivants : Tri à bulles (Bubble Sort) et Tri fusion (Merge Sort). Lequel choisiriez-vous pour trier une grande liste et pourquoi ? (4 points)

Réponse :

_____

_____

_____

_____

_____

_____

_____

Question 1.4 : Python et JavaScript (7 points)

a) [PYTHON] Qu'est-ce qu'une fonction en Python ? (4 points)

Voici un exemple de fonction qui additionne deux nombres :

```
def additionner (a, b):
    resultat = a + b
    return resultat

#Utilisation
```

```
Print(additionner((3, 5))     #Affiche 8
```

En vous inspirant de cet exemple, écrivez maintenant une fonction appelée `multiplier` qui prend deux nombres en paramètres et retourner leur produit

Réponse :

b) [JAVASCRIPT] Quelle est la différence entre `let`, `const` et `var` pour déclarer une variable en JavaScript ? Donnez un exemple pour chacun. (3 points)

Réponse :

PARTIE 2 : QCM (25 points)

Consigne : Cochez UNE SEULE réponse par question. (1,25 point par bonne réponse)

Section A : Python/JavaScript (12 questions)

Question 2.1

Quelle est la sortie du code Python suivant ?

```
x = [1, 2, 3]
y = x
y.append(4)
print(x)
```

- ☐ A) [1, 2, 3]
- ☐ B) [1, 2, 3, 4]
- ☐ C) [4]
- ☐ D) Erreur

Question 2.2

Quelle méthode permet de convertir une chaîne en majuscules en Python ?

- ☐A) string.toUpperCase()
- ☐ B) string.upper()
- ☐ C) string.capitalize()
- ☐ D) string.uppercase()

Question 2.3

Quelle est la sortie de ce code Python ?

```
liste = [1, 2, 3]
liste.append([4, 5])
print(len(liste))
```

- ☐ A) 3
- ☐ B) 4
- ☐ C) 5
- ☐ D) Erreur

Question 2.4

En JavaScript, que retourne "5" + 3 ?

- ☐ A) 8
- ☐ B) "53"
- ☐ C) "8"
- ☐ D) Erreur

Question 2.5

Quelle méthode Python permet de trier une liste sur place (in-place) ?

- ☐ A) sorted(list)
- ☐ B) list.sort()
- ☐ C) list.order()
- ☐ D) sort(list)

Question 2.6

En Python, que fait l'instruction continue dans une boucle ?

- ☐ A) Sort de la boucle
- ☐ B) Passe à l'itération suivante
- ☐ C) Redémarre la boucle depuis le début
- ☐ D) Génère une erreur

Question 2.7

Quelle est la complexité de list.append(element) en Python ?

- ☐ A) O(1)
- ☐ B) O(N)
- ☐ C) O(log N)
- ☐ D) $O(N^2)$

Question 2.8

En Python, comment créer une liste de 10 zéros ?

- ☐ A) [0] * 10
- ☐ B) list(0, 10)
- ☐ C) range(10)
- ☐ D) zeros(10)

Question 2.9

Que retourne bool("") en Python ?

- ☐ A) True
- ☐ B) False

- ☐ C) None
- ☐ D) Erreur

Question 2.10

En JavaScript, quelle syntaxe déstructure correctement un objet ?

- ☐ A) var {name, age} = person
- ☐ B) var [name, age] = person
- ☐ C) var name, age = person
- ☐ D) var (name, age) = person

Question 2.11

En Python, que fait "Hello World".split() ?

- ☐ A) Retourne "Hello"
- ☐ B) Retourne ["Hello", "World"]
- ☐ C) Retourne "HelloWorld"
- ☐ D) Génère une erreur

Question 2.12

En Python, que retourne `"Hello"[1:4]` ?

- ☐ A) `"ell"`
- ☐ B) `"Hel"`
- ☐ C) `"llo"`
- ☐ D) `"Hello"`

Section B : Git & GitHub (8 questions)

Question 2.13

Quelle commande Git permet de créer un nouveau dépôt local ?

- ☐ A) git create
- ☐ B) git new

- ☐ C) git init
- ☐ D) git start

Question 2.14

Quelle commande permet de voir l'état actuel des fichiers dans Git ?

- ☐ A) git state
- ☐ B) git status
- ☐ C) git check
- ☐ D) git info

Question 2.15

Que fait la commande git add . ?

- ☐ A) Ajoute un seul fichier
- ☐ B) Ajoute tous les fichiers modifiés
- ☐ C) Crée un nouveau fichier
- ☐ D) Supprime tous les fichiers

Question 2.16

Quelle commande permet de valider (commit) des changements avec un message ?

- ☐ A) git save "message"
- ☐ B) git commit -m "message"
- ☐ C) git push "message"
- ☐ D) git validate "message"

Question 2.17

Que fait la commande git push ?

- ☐ A) Télécharge les modifications depuis le dépôt distant
- ☐ B) Envoie les commits locaux vers le dépôt distant
- ☐ C) Crée une nouvelle branche
- ☐ D) Supprime le dépôt local

Question 2.18

Quelle commande permet de récupérer les modifications depuis le dépôt distant ?

- ☐ A) git download
- ☐ B) git fetch
- ☐ C) git pull
- ☐ D) git get

Question 2.19

Qu'est-ce qu'une branche (branch) dans Git ?

- ☐ A) Une copie du dépôt
- ☐ B) Une ligne de développement indépendante
- ☐ C) Un fichier de configuration
- ☐ D) Un type de commit

Question 2.20

Quelle commande permet de créer ET basculer vers une nouvelle branche ?

- ☐ A) git branch nouvelle-branche
- ☐ B) git switch nouvelle-branche
- ☐ C) git checkout -b nouvelle-branche
- ☐ D) git create-branch nouvelle-branche

PARTIE 3 : EXERCICES PRATIQUES (45 points)

Consigne : Vous pouvez répondre en Python OU en JavaScript au choix. Précisez votre langage.
Sauf si l'exercice exige un des deux langages ou les 2 langages.

Exercice 3.1 : Conversion de température (10 points)

Tiré du TP1 - Exercice 1.2

Écrivez un programme qui :

- Lit une température en degrés Celsius (nombre flottant)

- Convertit cette température en degrés Fahrenheit
- Affiche le résultat avec exactement 2 décimales

Formule : F = (C × 9/5) + 32

Exemple d'entrée :

25.5

Exemple de sortie :

77.90

Tests à effectuer :

- C = 0 → F = 32.00
- C = 100 → F = 212.00
- C = -40 → F = -40.00

Votre solution :

Langage choisi : _____

```
# Écrivez votre code ici
```

Grille de notation :

- Lecture de la température : 2 pts
- Application de la formule : 3 pts
- Formatage avec 2 décimales : 3 pts
- Gestion des cas limites : 2 pts

Exercice 3.2 : Année bissextile (12 points)

Tiré du TP1 - Exercice 2.5

Écrivez un programme qui détermine si une année est bissextile.

Règles :

- Une année est bissextile si elle est divisible par 4
- SAUF si elle est divisible par 100
- MAIS si elle est divisible par 400, elle est bissextile

Formule logique :

bissextile = (divisible par 4 ET non divisible par 100) OU (divisible par 400)

Exemples :

Entrée : 2000
Sortie : Bissextile
Raison : Divisible par 400

Entrée : 1900
Sortie : Non bissextile
Raison : Divisible par 100 mais pas par 400

Entrée : 2024
Sortie : Bissextile
Raison : Divisible par 4 et pas par 100

Entrée : 2023
Sortie : Non bissextile
Raison : Pas divisible par 4

Votre solution :

Langage choisi : _____

```
# Écrivez votre code ici
```

Grille de notation :

- Lecture de l'année : 1 pt
- Logique de la condition (divisible par 4) : 3 pts
- Exception pour 100 : 3 pts
- Exception pour 400 : 3 pts
- Affichage correct : 2 pts

Exercice 3.3 : Table de multiplication (10 points)

Tiré du TP1 - Exercice 3.6

Écrivez un programme qui :

- Lit un nombre entier N
- Affiche la table de multiplication de N de 1 à 10
- Format : "N x i = résultat"

Exemple pour N = 7 :

7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70

Contrainte : Utilisez une boucle for

Votre solution :

Langage choisi : _____

```
# Écrivez votre code ici
```

Grille de notation :

- Lecture de N : 2 pts
- Boucle for correcte (1 à 10) : 3 pts
- Calcul du produit : 2 pts
- Formatage de l'affichage : 3 pts

Exercice 3.4 : EXERCICE GLOBAL - Débogage d'un analyseur de liste (13 points)

*Exercice de débogage et correction de code — **Langage obligatoire : Python***

Le programme suivant est censé analyser une liste de nombres entiers, mais il contient **plusieurs erreurs**.

Votre mission : Identifiez et corrigez toutes les erreurs pour que le programme fonctionne correctement.

Fonctionnalités requises :

1. **Lecture des données** :
   - Lire un nombre N (taille de la liste)
   - Lire N nombres entiers
2. **Calculs statistiques** :
   - Calculer la somme totale
   - Calculer la moyenne (2 décimales)
   - Trouver le minimum
   - Trouver le maximum
   - Compter les nombres pairs
   - Compter les nombres impairs
3. **Affichage formaté** :

   ```
   === ANALYSE DE LA LISTE ===
   Nombre d'éléments: X
   Somme: Y
   Moyenne: Z.ZZ
   Minimum: A
   Maximum: B
   Nombres pairs: C
   Nombres impairs: D
   ```

```
### **Exemple de test :**

**Entrée :**
```
6
12 7 23 8 15 4
```

**Sortie attendue :**
```
=== ANALYSE DE LA LISTE ===
Nombre d'éléments: 6
Somme: 69
Moyenne: 11.50
Minimum: 4
Maximum: 23
Nombres pairs: 3
Nombres impairs: 3
```
```

CONSIGNES :

1. **Identifiez les erreurs** (au moins 7 erreurs dans ce code sachant qu'il y'en a 10) (7 pts)
2. **Corrigez et écrivez le code** pour qu'il fonctionne correctement dans l'espace réponse ci-dessous (5 pts)
3. **Testez** avec l'exemple fourni

Contraintes :

- N'utilisez PAS les fonctions `sum()`, `min()`, `max()` de Python
- Implémentez les calculs avec des boucles

GREEN UP ACADEMY

CODE AVEC ERREURS À CORRIGER :

```
# Programme d'analyse de liste (CONTIENT DES ERREURS !)

# Lecture des données
n = input("Nombre d'éléments: ")
print("Entrez les nombres séparés par des espaces:")
nombres = input().split()

# Initialisation des variables
somme = 0
minimum = 0
maximum = 0
pairs = 0
impairs = 0

# Calculs
for i in range(n):
    nombre = nombres[i]

    # Somme
    somme = somme + nombre

    # Min et Max
    if nombre < minimum:
        minimum = nombre
    if nombre < maximum:
        maximum = nombre

    # Pairs et impairs
    if nombre % 2 = 0:
        pairs = pairs + 1
    else:
        impairs = pairs + 1

# Calcul de la moyenne
moyenne = somme / len(nombres)

# Affichage
print("=== ANALYSE DE LA LISTE ===")
print("Nombre d'éléments:", len(nombres))
print("Somme:", somme)
print("Moyenne:", moyenne)
print("Minimum:", minimum)
print("Maximum:", maximum)
print("Nombres pairs:", pairs)
print("Nombres impairs:", impairs)
```

Votre solution :

```
# Écrivez votre code ici
```

BARÈME GLOBAL

Partie	Points	Pourcentage
Partie 1 : Questions de cours	/30	30%
Partie 2 : QCM	/25	25%
Partie 3 : Exercices pratiques	/45	45%
TOTAL	**/100**	**100%**

ÉCHELLE DE NOTATION

Note /100	Note /20	Appréciation
90-100	18-20	Excellent - Maîtrise complète
80-90	16-18	Très bien - Très bonne maîtrise
70-80	14-15	Bien - Bonne maîtrise
60-70	12-13	Assez bien - Maîtrise satisfaisante
50-60	10-11	Passable - Connaissances de base
< 50	< 10	Insuffisant - Lacunes importantes

CONSEILS MÉTHODOLOGIQUES

Pour les questions de cours :

1. Lisez attentivement la question
2. Structurez votre réponse (définition → explication → exemple si pertinent)
3. Soyez précis et concis
4. Utilisez le vocabulaire technique approprié

Pour le QCM :

1. Lisez toutes les réponses avant de cocher
2. Éliminez les réponses évidemment fausses
3. En cas de doute, fiez-vous à votre première impression
4. Ne cochez qu'UNE SEULE réponse

Pour les exercices pratiques :

1. **Lisez l'énoncé complet** avant de coder
2. **Identifiez** les entrées et sorties
3. **Planifiez** votre algorithme (pseudo-code mental ou sur brouillon)
4. **Codez** étape par étape
5. **Testez** avec les exemples fournis
6. **Vérifiez** les cas limites

Cas limites à considérer :

- Valeurs minimales (0, 1)
- Valeurs négatives
- Listes vides
- Valeurs très grandes
- Chaînes vides

RAPPELS IMPORTANTS

Opérateurs Python essentiels :

- `//` : Division entière
- `%` : Modulo (reste de la division)
- `**` : Puissance
- `and`, `or`, `not` : Opérateurs logiques

Boucles :

Boucle for avec range

```
# Boucle for avec range
for i in range(1, 11):  # De 1 à 10 inclus
    print(i)

# Boucle while
```

```python
while condition:
    # instructions
```

Formattage d'affichage :

```python
# Avec f-string (Python 3.6+)
print(f"Résultat: {valeur:.2f}")  # 2 décimales

# Avec format()
print("Résultat: {:.2f}".format(valeur))
```

Bon courage !

"La programmation, c'est l'art de transformer une idée en instructions que la machine peut exécuter."